

¿Cómo es un Proceso de Decisión de Markov (MDP)?

$s \in S$

$a \in A$

$A_k: S \rightarrow P(A)$

$T: S \times A \times S \rightarrow \mathbb{R}$ tal que $T(s, a, s') = \Pr(S_{t+1} = s' | S_t = s, A_t = a)$
también queremos saber que tan conveniente es cambiar de estado.

$r: S \times A \times S \rightarrow \mathbb{R}$ tal que $r(s, a, s')$ es la recompensa de ir de s a s' con la acción a

$S_f \subseteq S$ Conjunto de estados terminales
 $0 \leq \gamma < 1$ factor de descuento

Quiero encontrar una política

$\pi: S \times A \rightarrow \mathbb{R}$

$\pi(s, a) = \Pr(A_t = a | S_t = s)$

tal que

$E^\pi[R_t]$ sea máxima

$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = r_t + \gamma R_{t+1}$

$s: V^\pi(s) = E^\pi[R_t | S_t = s]$

$\pi_1 \leq \pi_2$ si

$V^{\pi_1}(s) \leq V^{\pi_2}(s) \quad \forall s \in S$

Para saber si podemos calcular políticas

$$V^\pi(s) = \sum_{a \in A_t(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^\pi(s')]$$

$$Q^\pi(s, a) = E^\pi[R_t | S_t = s, A_t = a]$$

$$Q^\pi(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma \sum_{a' \in A(s')} \pi(s', a') Q^\pi(s', a')]$$

$V^\pi(s')$

$$V^\pi(s) = \sum_{a \in A(s)} \pi(s, a) Q(s, a)$$

$$\pi \in \pi^* \quad V^\pi \in \pi$$

π^* es la política óptima
 V^* y Q^* son las funciones de valor óptimas

$$V^*(s) = V^{\pi^*}(s), \quad Q^*(s, a) = Q^{\pi^*}(s, a)$$

$$V^*(s) = \sum_{a \in A(s)} \pi(s, a) Q^*(s, a)$$

ya no es esto
 es:

$$V^*(s) = \max_{a \in A(s)} Q^*(s, a)$$

$$\pi(s, a) = \begin{cases} 1 & \text{si } a = \arg \max_{a \in A(s)} Q(s, a) \\ 0 & \text{o.c.} \end{cases}$$

entonces

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma \max_{a' \in A(s')} Q^*(s', a')]$$

Eq. Optimalidad de Bellman.


```

def iter_valor(MDP, max_iter = 1e5, tol = 1e-6):
    Q = {}
    for s in MDP.estados():
        for a in MDP.acciones_legales(s):
            Q[(s,a)] = 0
    for _ in range(max_iter):
        err = 0
        for (s,a) in Q.keys():
            q = Sum(MDP.T(s,a,s') * (MDP.r(s,a,s')
                + MDP.gamma * max(Q[(s',a')] for a'
                    in MDP.acciones_legales(s'))
                for s' in MDP.estados())
            err = max(err, abs(Q[(s,a)] - q))
            Q[(s,a)] = q
        if err < tol:
            break

```

```

def genera_politica_greedy(MDP, Q):
    pi = {}
    for s in MDP.estados():
        pi[s] = max(a for a in MDP.acciones_legales(s),
            key=lambda a: Q[(s,a)])

```

Black Jack

Estados:

$S = (m_1, n_1, a_1, a_2, d_1, n_2, a_3, d_2)$
 ↙ ↘ ↙ ↘ ↙ ↘
 Número Número as as doblar
 mio Oponente mio Oponente

As = 11 0 1

~~S: llega a 21~~

Factor de descuento: 0 Porque es continuo

MDP

$$S = \{s_1, \dots, s_n\}$$

$$A = \{a_1, \dots, a_n\}$$

$$A_t: S \rightarrow P(A)$$

$$S_t \in S \text{ estados terminales}$$

$$T: S \times A \times S \rightarrow \mathbb{R} \quad T(s, a, s') = \Pr(S_{t+1}=s' | S_t=s, A_t=a)$$

$$r: S \times A \times S \rightarrow \mathbb{R} \quad r(s, a, s')$$

$$r: r \in [0, 1]$$

$$\sum_{s' \in S} T(s, a, s') = 1$$

$$\pi: S \times A \rightarrow \mathbb{R} \quad \pi(s, a) = \Pr(A_t=a | S_t=s) \quad \sum_{a \in A(s)} \pi(s, a) = 1$$

$$\pi^* = \arg \max_{\pi} E^{\pi}[R_T] \quad \forall s \in S$$

$$E^{\pi}[r_t + \gamma R_{t+1} + \gamma^2 r_{t+2} + \dots] \quad \forall s \in S$$

$$E^{\pi}[r_t + \gamma R_{t+1}] \quad \forall s \in S$$

$$V^{\pi}(s) = E^{\pi}[r_t + \gamma R_{t+1}] = \sum_{a \in A(s)} \pi(s, a) \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^{\pi}(s')]$$

Ecuación Optimalidad de Bellman

$$V^*(s) = \max_a \left\{ \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^*(s')] \right\}$$

$$\pi^*(s, a) = \begin{cases} 1 & \text{si } a = \arg \max_{a'} \sum_{s' \in S} T(s, a', s') [r(s, a', s') + \gamma V^*(s')] \\ 0 & \text{o.c.} \end{cases}$$

MDP sim

$$S = \{s_1, \dots, s_n\}$$

$$A = \{a_1, \dots, a_n\}$$

$$A_t: S \rightarrow P(A)$$

$$S_t \in S$$

$$s' = \text{sucesor}(s, a)$$

$$r: S \times A \times S \rightarrow \mathbb{R} \quad r(s, a, s')$$

$$r: r \in [0, 1]$$

¿Cómo evaluar una política π con MDPs?

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} \sum_{s' \in S} T(s, a, s') [r(s, a, s') + \gamma V^\pi(s')]$$

Como que conozco $\hat{V}^\pi(s)$ y estoy en el estado s

escojo a a partir de π

aplico a a MDPsim y obtengo s' y r

$s' = \text{sucesor}(s, a)$

$r = r(s, a, s')$

entonces $\hat{V}_{t+1}^\pi(s) = r + \gamma \hat{V}_t^\pi(s')$

$$\hat{V}^\pi(s) \leftarrow \hat{V}^\pi(s) + \alpha [\hat{V}_{t+1}^\pi(s) - \hat{V}^\pi(s)]$$

Diferencia temporal

π determinista
 $a = \pi(s)$

π estocástico
 $\text{umbral} = \text{random.rand}()$

nivel = 0

for a in $A(s)$

nivel += $\pi(s, a)$

if $\text{umbral} \leq \text{nivel}$:

return a

raise $\text{ValueError}()$

class MDPsim:

def __init__(self, s, r):

self.s = s

self.r = r

def acciones_legales(self, s):

return iterable con acciones legales en s

def sucesor(self, s, a):

return r

def terminal(self, s):

return True/False


```

class Poltica
    def __init__(self, P = dict):
        self.P = P

```

```

    def __call__(self, s):
        return self.P[s]

```

```

    def estado_inicial(self):
        return estado

```

ya tengo $\hat{V}^\pi(s)$ Para un estado s

Aplico:

$a = \text{Poltica}(s)$

$s' = \text{MDPsim.sucesor}(s, a)$

$r = \text{MDPsim.recompensa}(s, a, s')$

$v = r + \text{MDPsim} \cdot \gamma \cdot \hat{V}^\pi(s')$

$\hat{V}^\pi(s) = \hat{V}^\pi(s) + \alpha [v - \hat{V}^\pi(s)]$

```

def TD_0(mdl,  $\pi$ ,  $\alpha$ , max_episodios, max_it)

```

$v = \{0 \text{ for } s \text{ in } \text{MDL.S}\}$

for _ in range(max_episodios)

err = 0

$s = \text{mdl.estado_inicial}()$

for _ in range(max_it):

$a = \pi(s)$

$s' = \text{mdl.sucesor}(s, a)$

$r = \text{mdl.sucesor}(s, a, s')$

$v = v[s] - \alpha * (r - \text{mdl.r} = v[s'] - v[s])$

err = max(err, abs(v - v[s]))

$v[s], s = v, s'$

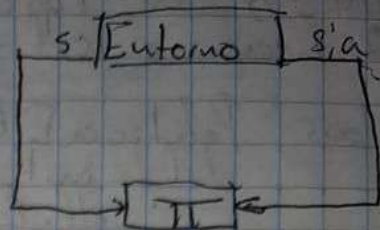
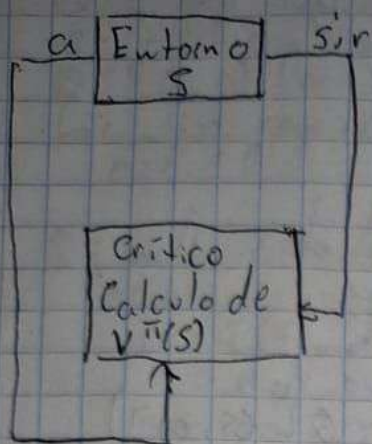
if mdl.terminal(s)

break

if err < tol:

break

return v



$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + r \sum_{a' \in A_L(s')} \pi(s', a') Q(s', a')]]$$

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + r \max_{a' \in A_L(s)} Q^*(s', a')]]$$

Tengo $Q(s, a)$

Aplico:

$s' = \text{mdl. sucesor}(s, a)$

$r = \text{mdl. recompensa}(s, a, s')$

$\begin{cases} a' = \arg \max_{a'} Q(s', a') \\ a' = \text{política}(s') \end{cases}$

$q = r + \text{mdl. } \gamma * Q(s', a')$

$Q(s, a) = Q(s, a) + \alpha (q - Q(s, a))$

Tengo $\hat{V}^\pi(s)$ para un estado s

Aplico:

$s' = \text{MDPsim. sucesor}(s, a)$

$r = \text{MDPsim. recompensa}(s, a, s')$

$V = r + \text{MDPsim. } \gamma * \hat{V}^\pi(s')$

$$\hat{V}^\pi(s) = \hat{V}^\pi(s) + \alpha [V - \hat{V}^\pi(s)]$$

$$\pi(s) = \arg \max_{a \in A_L(s)} Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') [r(s, a, s') + r \sum_{a' \in A_L(s')} \pi(s', a') Q(s', a')]]$$

$$Q^*(s, a) = \sum_{s' \in S} T(s, a, s') + r \max_{a' \in A_L(s)} Q^*(s', a')]$$

SARSA: on-policy
Q-Learning: off-policy.

```
class PolíticoE Greedy (Político):  
    def __init__(self, E):  
        self.E = E
```

```
    def __call__(s, Q):  
        if random.random() < E:  
            return random.choice(A)  
        return max(A, key = lambda a: Q[s, a])
```

```
def SARSA (mdl, num_ep, num_it,  $\alpha$ , tol, E)  
     $\pi$  = PolíticoE Greedy(E)  
    Q = {}  
    for s in S:  
        for a in mdl.acciones legales(s):  
            for _ in range(num_ep):  
                err = 0  
                s = mdl.estado inicial()  
                a =  $\pi$ (s, mdl.acciones legales(s), Q)  
                for _ in range(num_it):  
                    s' = mdl.sucesor(s, a)  
                    r = mdl.recompensa(s, a, s')  
                    a' = Político(s', mdl.acciones legales(s'), Q)  
                     $q_{err} = \alpha * r + (1 - \alpha) * Q[s, a]$   
                    err = max(err, abs(Q[s, a] -  $q_{err}$ ))  
                    s, a = s', a'  
                if mdl.terminal(s):  
                    break  
            if err < tol:  
                break  
    return Q
```



```

def Qlearning (mdl, num-ep, nom-it,  $\alpha$ , tol, e):
     $\pi = \text{PoliticoyEGreedy}(e)$ 
     $Q = \{ (s, a) \text{ for } s \text{ in mdl.s for } a \text{ in mdl.acciones-legales}(s) \}$ 
    for _ in range (num-ep):
        err = 0

        s = mdl.estado-inicial()
        for _ in range (nom-it):
            a =  $\pi(s, \text{mdl.acciones-legales}(s), Q)$ 
            s' = mdl.sucesor(s, a)
            r = mdl.recompensa(s, a, s')
             $q = r + \alpha * \max_{a'} (Q[s', a'])$  for  $a'$  in mdl.acciones-legales(s')
            err = max(err, abs(q, Q[s, a]))
            s = s'
            if mdl.terminal(s):
                break
        if err < tol:
            break
    return Q

```

Optimización

(Búsquedas locales)

$f: X \rightarrow \mathbb{R}$

y lo que quiero es encontrar x^* tal que

$$f(x^*) = \max_{x \in X} f(x)$$

$$f(x^*) = \min_{x \in X} f(x)$$

$$A = \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ a_{21} & & & \\ \vdots & & \ddots & \\ a_{m1} & & & 0 \end{bmatrix}$$

lo que quiero encontrar es un circuito cerrado

$x = \text{Permutación de } [1, \dots, n]$

donde

$$f(x) = \sum_{i=1}^{n-1} a_{x[i], x[i+1]} + a_{x[n], x[1]}, x \in \mathbb{Z}$$

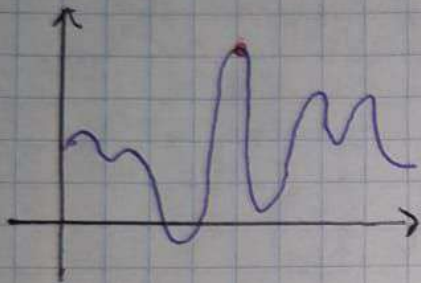
$$\begin{bmatrix} 1, 2, 3, 4, 5, 6, 7, 8, 9 \\ 10, 11, 12, 13, 14, 15, 16, 17, 18 \end{bmatrix}$$

$$y \ x^* = \min_x f(x)$$

$$f: X \rightarrow \mathbb{R}$$

Un máximo global es:

$$x^* = \arg \max_{x \in X} f(x)$$



$$x'_1 = \lim_{\Delta \rightarrow 0} x_1 - \Delta$$

$$x'_2 = \lim_{\Delta \rightarrow 0} x_1 + \Delta$$

Tablero 4x4

1			
	2		
3			
		4	

$\begin{matrix} (1, 2, 3, 4) \\ (2, 1, 3, 4) \\ (3, 2, 1, 4) \\ (1, 3, 2, 4) \end{matrix} \left. \vphantom{\begin{matrix} (1, 2, 3, 4) \\ (2, 1, 3, 4) \\ (3, 2, 1, 4) \\ (1, 3, 2, 4) \end{matrix}} \right\} \text{Permutaciones}$

Debemos de poner n reinas
Pero no pueden comerse entre
si. Entonces, debe haber una en
cada renglón

Si existe una topología en X
Un máximo local es:

$$x^* = \arg \max_{x \in \text{vecinos}(x)} f(x)$$

Cuando el gradiente es cero, es un máximo, mínimo o una "silla"

Inicializa $x \in X$ en forma aleatoria
e $\max \leftarrow f(x)$

Para max, epoch iteraciones

Lo implementaremos minimizando el costo.


```

class Pb ORim():
    def costo(self, x):
        if not self.es_estado(x):
            raise ValueError("")
        return Px

    def es_estado(self, x):
        return True/False

    def estado_aleatorio(self):
        return estado

    def vecinos(self, x):
        return iterable con los estados vecinos de x

    def vecino_aleatorio(self, x):
        return un vecino de x

def Búsqueda Aleatoria (Pb, num_iter):
    x = Pb.estado_aleatorio()
    c = Pb.costo(x)
    for _ in range (num_iter):
        otro = Pb.estado_aleatorio()
        c_otro = Pb.costo(otro)
        if c_otro < c:
            x, c = otro, c_otro

```

Con esto modificamos la búsqueda aleatoria... Para encontrar los mínimos locales

```

def descenso_collinas (Pb, num_iter):
    x = Pb.estado_aleatorio()
    c = Pb.costo(x)
    for _ in range (num_iter):
        termina = True
        for vecino in Pb.vecinos(x):
            c_vecino = Pb.costo(vecino)
            if c_vecino < c:
                x, c = vecino, c_vecino
                termina = False
        if termina:
            break
    return x

```


def Simulador(Pb, T_{max}, calendarizador, T_{min}):

x = Pb.estado_aleatorio()

c = Pb.costo

T = T_{max}

while (T > T_{min})

vecino = Pb.vecino_aleatorio(x)

c_vecino = Pb.costo(vecino)

inc = c - c_vecino

if inc >= 0 or random.random() < math.exp(inc/T)

x, c = vecino, c_vecino

T = calendarizador(T, i)

i += 1

return x, c

$\frac{T_{max}}{i+1}$

def Calendarizador(T_{max}, i):

return $\frac{T_{max}}{\ln(\ln(i+1)+1)}$

$\frac{T_{max}}{\ln(\ln(i+1)+1)}$

$\frac{T_{max}}{\ln(\ln(i+1)+1)}$

Población: $\{x_1, x_2, \dots, x_n\}$ $x_i \in X$

Capacidad: $[V_{i1}, V_{i2}, \dots, V_{in}]$

adaptación: $X \rightarrow \mathbb{R}^+$ adaptación(x_i) (Inversamente proporcional al costo)

$$\text{adaptación}(x_i) = \frac{1}{\text{costo}(x_i)}$$

$$= \frac{K - \text{costo}(x_i)}{\max(0, K - \text{costo}(x_i))}$$

Selección: - ruleta
- torneo

Cruza $[V_{i1}, V_{i2}, \dots, V_{in}]$

$[V_{j1}, V_{j2}, \dots, V_{jn}]$

$[V_{k1}, V_{k2}, \dots, V_{kn}]$

Mutación
Elitismo